School of Future Tech

Case Study Report

on

Case Study 54.File Compression & Decompression System

by

Name: Hazel Lakshmana
Roll no.: 15009672515

Index:

1.	Introduction to the Case Study
2.	Case Background (Abstract)
3.	Case Study Design
4.	Methods & Algorithms Technology Applied
5.	Implementation Details and Snapshots
6.	Results and Conclusion
7.	References

1. Introduction to the Case Study

With the rapid growth of digital data, efficient storage and data transmission have become essential. Large files require more disk space and take longer to transfer across networks. File compression techniques help reduce file size while preserving the original information, making storage and transmission more efficient.

This case study focuses on designing and implementing a **File Compression and Decompression System using C++**. The system reads text files, compresses them using a simple algorithm, and later restores them back to their original form through decompression.

The project demonstrates how file handling and basic algorithms can be combined to create a practical utility for reducing file sizes. It also highlights the importance of efficient data representation and processing when working with large datasets.

2. Problem Statement / Case Background (Abstract)

Background

In many computing applications, files often contain repeated characters or patterns. Storing these repetitions directly results in larger file sizes and inefficient use of storage resources.

Traditional file storage systems store data exactly as it appears, without considering patterns or repetitions that could be compressed. Compression techniques can reduce file size by representing repeated sequences in a shorter format.

Abstract

This case study presents the design and implementation of a **File Compression and Decompression System using C++**. The system utilizes a simple compression technique known as **Run-Length Encoding (RLE)**. In this method, consecutive repeated characters in a file are replaced with a single character followed by the number of repetitions.

For example:

aaaaabbbbcc

is compressed to:

a5b4c2

The program reads an input file, compresses the data using this technique, and writes the compressed result to a new file. The decompression function reads the

compressed file and reconstructs the original data. This system demonstrates how basic algorithms and file handling operations in C++ can be applied to solve practical data storage problems.

3. Case Study Design

The **File Compression and Decompression System** is designed as a simple modular program consisting of the following stages:

File Input Handling

The program begins by asking the user to enter the input file name and output file name. It uses file streams to read data from the input file.

Compression Process

The compression module reads characters from the file and counts consecutive repeated characters. Instead of writing repeated characters multiple times, it stores the character followed by its frequency.

Decompression Process

The decompression module reads the compressed file, extracts the character and its count, and recreates the original sequence of characters.

Output Generation

The final result is saved as a new file. Depending on the user's choice, the program either produces a compressed file or restores the original data through decompression.

This design ensures clear separation between input handling, compression logic, and decompression logic.

4. Methods & Algorithms Technology Applied

Key Methods Used

File Handling

The program uses C++ file streams:

- **ifstream** to read data from files
- **ofstream** to write processed data to output files

These classes enable efficient input and output operations.

Run-Length Encoding (RLE)

Run-Length Encoding is a simple data compression algorithm that replaces consecutive repeated characters with a character and its count.

Example:

Input: aaaabbbcc

Output: a4b3c2

This technique works best for files with many repeated characters.

Compression Algorithm Steps

1. Read the first character from the file.
2. Compare it with the next character.
3. If both characters are the same, increase the count.
4. If they are different, write the character and its count to the output file.
5. Continue until the entire file is processed.

Decompression Algorithm Steps

1. Read a character and the number associated with it.
2. Repeat writing the character to the output file according to the count.
3. Continue until the compressed file is completely processed.

Technology Stack Used

Programming Language:
C++

Libraries Used:

- iostream (for input and output operations)

- fstream (for file handling)
- string (for handling file names and data)

Development Environment:

- VS Code / Any C++ IDE
- Command Line Compiler (g++ or clang++)

Data Storage:

- Text files used as input and output data.

5. Implementation Details and Snapshots.

The implementation is written in **C++** and organized around a class called **FileCompressor** which contains two main functions:

compressFile()

This function reads the input file, applies the Run-Length Encoding compression algorithm, and writes the compressed data to the output file.

decompressFile()

This function reads the compressed file, extracts the characters and their counts, and reconstructs the original data.

Main Program

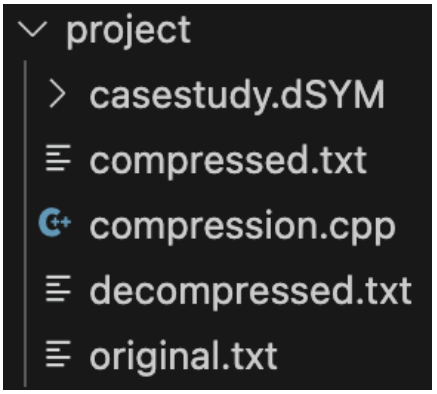
The main function provides a simple menu where the user can choose between:

1. Compress File
2. Decompress File

Based on the user's choice, the appropriate function is executed.

Screenshots of the program execution, source code, input files, compressed output, and decompressed output are included to demonstrate the working of the system.

Project Folder Structure

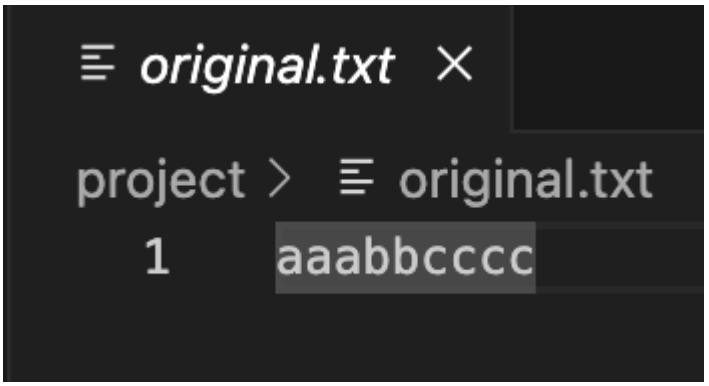


Source Code

project > compression.cpp > FileCompressor > compressFile(string, string)

```cpp
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

// Class for compression and decompression
class FileCompressor
{
public:

    // Function to compress file
    void compressFile(string inputFile, string outputFile) ...

    // Function to decompress file
    void decompressFile(string inputFile, string outputFile) ...
};

int main() ...
```

Input File



Program Running (Compression)

```
---- File Compression System ----
1. Compress File
2. Decompress File
Enter your choice: 1
Enter input file name: original.txt
Enter output file name: compressed.txt
File compressed successfully.
```

Compressed File Output

compressed.txt

project > compressed.txt

```
1    a3b2c4
```

Program Running (Decompression)

```
Enter your choice: 2
Enter input file name: compressed.txt
Enter output file name: decompressed.txt
File decompressed successfully.
```

Decompressed File

decompressed.txt

project > decompressed.txt

```
1    aaabbcccc
```

6. Results and Conclusion.

The implemented system successfully compresses files containing repeated characters using the Run-Length Encoding technique. The compressed file occupies less space compared to the original file when there are many repeated patterns.

The decompression module accurately restores the original data without losing information, demonstrating that the compression process is reversible.

Conclusion

This case study demonstrates the implementation of a simple file compression and decompression system using C++. The system integrates file handling operations with a basic compression algorithm to reduce file size efficiently.

Although Run-Length Encoding is a simple compression technique, it effectively illustrates how algorithms can optimize data storage. The project provides a foundation for exploring more advanced compression techniques such as Huffman Coding or Lempel-Ziv algorithms in future implementations.

7. References

- C++ official documentation for file handling and standard libraries.
- Documentation for iostream and fstream libraries used for input and output operations.
- Tutorials and articles related to Run-Length Encoding and data compression techniques.
- Online C++ programming resources and textbooks explaining file processing and algorithms.